



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Летучка № 7

по курсу «Языки и методы программирования»

«Функциональный подход. $\text{Roots}\langle T \rangle$. GUI-построение траектории»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Функциональные интерфейсы	1
Метод map()	2
Задание 1: Квадратное уравнение	2
Класс Point	2
Класс Roots	3
Главный класс Stone	4
Задание 2: Полиномы	4
Метод eval()	4
Метод of() для полиномов	5
Метод toPoints()	6
Тестирование	6
Задание 3: GUI Построение траектории	6
Класс Plot	7
Панель рисования TrajectoryPanel	8
Makefile	9
Запуск	9
Вывод	9

Цель работы

Изучить функциональный подход в Java с использованием интерфейсов Function и Consumer. Реализовать универсальный класс Roots для нахождения корней уравнений. Создать GUI-приложение для визуализации траектории брошенного камня.

Задачи:

- Изучить функциональные интерфейсы Java.
- Реализовать класс Roots с методами map() и forEach().
- Найти корни квадратного уравнения для траектории камня.
- Создать GUI для построения траектории.

Теоретические сведения

Функциональные интерфейсы

Java предоставляет встроенные функциональные интерфейсы в пакете java.util.function:

- Function<T, R> — преобразование объекта T в объект R
- Consumer<T> — обработка объекта без возврата значения
- Supplier<T> — создание объекта
- Predicate<T> — проверка условия

Метод map()

Метод map() применяет функцию к каждому элементу коллекции, возвращая новую коллекцию с преобразованными элементами.

Задание 1: Квадратное уравнение

Класс Roots реализует функциональный подход для работы с корнями уравнений.

Класс Point

Point.java

```
public class Point {
    private double x, y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double dist() {
        return Math.sqrt(x * x + y * y);
    }

    public String toString() {
        return String.format("(%.2f, %.2f)", x, y);
    }
}
```

Класс Roots

Roots.java

```
public class Roots<T> {
    private HashSet<T> container;

    private Roots(HashSet<T> container) {
        this.container = container;
    }

    public static Roots<Double> of(double a, double b, double c,
double eps) {
        HashSet<Double> roots = new HashSet<>();
        if (a == 0.0) {
            if (b != 0.0) {
                roots.add(-c / b);
            }
        } else {
            double d = b * b - 4 * a * c;
            if (d >= 0.0) {
                if (d < eps)
                    d = 0.0;
                roots.add((-b + Math.sqrt(d)) / (2 * a));
                roots.add((-b - Math.sqrt(d)) / (2 * a));
            }
        }
        return new Roots<>(roots);
    }

    public <R> Roots<R> map(Function<T, R> f) {
        HashSet<R> c = new HashSet<>();
        for (T t : container) {
            c.add(f.apply(t));
        }
        return new Roots<>(c);
    }

    public void forEach(Consumer<T> f) {
        for (T t : container) {
            f.accept(t);
        }
    }
}
```

Главный класс Stone

Stone.java

```
public class Stone {
    private static final double G = 9.81;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double v = sc.nextDouble();
        double fi = sc.nextDouble() * Math.PI / 180;
        double h = sc.nextDouble();

        Roots.of(-G / 2, v * Math.sin(fi), -h, 1e-10)
            .map(t → new Point(t * v * Math.cos(fi), h))
            .map(pt → pt.dist())
            .forEach(x → System.out.println(x));
    }
}
```

Вычисление траектории камня: Вычисление траектории камня:

- Уравнение: $h + v \sin(\phi) t - \frac{g}{2} t^2 = 0$
- Решения — моменты времени, когда камень на высоте $h = 0$ (начало и конец полета)
- После нахождения t : координаты $x = v \cos(\phi) t$, $y = h$

Задание 2: Полиномы

Расширенный класс Roots для нахождения корней полиномов.

Метод eval()

Вычисление значения полинома $A_0x^n + A_1x^{n-1} + \dots + A_n$:

Roots.java

```
private static double eval(double[] A, double x) {
    double result = 0.0;
    for (double a : A) {
        result = result * x + a;
    }
    return result;
}
```

Это схема Горнера.

Метод of() для полиномов

Roots.java

```
public static Roots<Double> of(double[] A, double step, double eps)
{
    HashSet<Double> roots = new HashSet<>();
    // поиск корней на интервале
    double maxCoef = Math.abs(A[0]);
    for (double c : A) {
        if (maxCoef < Math.abs(c)) {
            maxCoef = Math.abs(c);
        }
    }

    double leftBound = -maxCoef - 1;
    double rightBound = -leftBound;
    double previous = eval(A, leftBound);

    for (double t = leftBound; t ≤ rightBound; t += step) {
        double current = eval(A, t);
        // проверка смены знака
        if (sign(previous) ≠ sign(current)) {
            // метод половинного деления
            double l = t - step;
            double r = t;
            while (r - l > eps) {
                double m = l + (r - l) / 2;
                if (sign(eval(A, m)) ≠ sign(eval(A, l))) {
                    r = m;
                } else {
                    l = m;
                }
            }
            roots.add((l + r) / 2);
        }
        previous = current;
    }
    return new Roots<>(roots, A);
}
```

Метод toPoints()

Roots.java

```
public Roots<Point> toPoints() {
    HashSet<Point> c = new HashSet<>();
    for (T t : container) {
        c.add(new Point((Double) t, eval(A, (Double) t)));
    }
    return new Roots<>(c, A);
}
```

Тестирование

Test.java

```
public class Test {
    public static void main(String[] args) {
        //  $5x^4 - 2x^2 + 3x - 9 = 0$ 
        Roots.of(new double[] { 5, 0, -2, 3, -9 }, 1e-2, 1e-2)
            .toPoints()
            .forEach(x → System.out.println(x));

        //  $x^6 - 64 = 0$ 
        Roots.of(new double[] { 1, 0, 0, 0, 0, 0, -64 }, 1e-5, 1e-9)
            .forEach(x → System.out.println(x));
    }
}
```

Задание 3: GUI Построение траектории

GUI-приложение для визуализации траектории брошенного камня.

Класс Plot

Plot.java

```
public class Plot extends JFrame {
    private JSpinner gSpinner;
    private JSpinner angleSpinner;
    private JSpinner velocitySpinner;
    private JSpinner heightSpinner;
    private TrajectoryPanel trajectoryPanel;

    public Plot() {
        setTitle("Stone Trajectory");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        JPanel controls = new JPanel(new GridLayout(2, 4, 10, 5));
        // элементы управления
        gSpinner = createSpinner(9.81, 0.1, 50.0, 0.1);
        angleSpinner = createSpinner(45.0, 0.0, 90.0, 1.0);
        velocitySpinner = createSpinner(10.0, 0.1, 100.0, 0.5);
        heightSpinner = createSpinner(0.0, 0.0, 100.0, 1.0);

        add(controls, BorderLayout.NORTH);
        trajectoryPanel = new TrajectoryPanel();
        add(trajectoryPanel, BorderLayout.CENTER);
    }
}
```


Панель рисования TrajectoryPanel

Plot.java

```
private class TrajectoryPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;

        double G = ((Number) gSpinner.getValue()).doubleValue();
        double angleDeg = ((Number)
angleSpinner.getValue()).doubleValue();
        double v = ((Number) velocitySpinner.getValue()).doubleValue();
        double h = ((Number) heightSpinner.getValue()).doubleValue();

        double fi = angleDeg * Math.PI / 180;
        double a = -G / 2;
        double b = v * Math.sin(fi);
        double c = -h;

        double d = b * b - 4 * a * c;
        double flightTime = 0;
        if (d ≥ 0) {
            double t1 = (-b + Math.sqrt(d)) / (2 * a);
            double t2 = (-b - Math.sqrt(d)) / (2 * a);
            flightTime = Math.max(Math.max(t1, t2), 0);
        }

        // рисование траектории
        Path2D path = new Path2D.Double();
        for (double t = 0; t ≤ flightTime; t += flightTime / 200) {
            double x = v * Math.cos(fi) * t;
            double y = h + v * Math.sin(fi) * t - G / 2 * t * t;
            // добавление точек
        }
        g2.draw(path);
    }
}
```

Makefile

Makefile

```
.PHONY: all clean run

all:
    javac -cp lib/flatlaf-3.7.jar -d out src/*.java

run:
    java -cp out:lib/flatlaf-3.7.jar Plot

clean:
    rm -f out/*.class
```

Запуск

terminal

```
$ make
$ make run
```

- G (m/s²) — ускорение свободного падения
- Angle (°) — угол запуска
- Velocity (m/s) — начальная скорость
- Height (m) — начальная высота

И траектория камня отображается на холсте.

Вывод

В ходе лабораторной работы были реализованы:

1. Функциональный класс Roots — универсальный контейнер для корней уравнений с методами `map()` и `forEach()`
2. Решение квадратного уравнения — нахождение корней для вычисления траектории камня
3. Решение полиномов — численный метод для нахождения корней произвольного полинома
4. GUI-приложение Plot — визуализация траектории брошенного камня с использованием FlatLaF и Swing

Функциональный подход в Java позволяет писать лаконичный код с использованием лямбда-выражений и цепочек методов.